

神经网络史话

从感知机到 ChatGPT

全 33 节

目录

楔子 一条线

第一章 初啼

第二章 界碑

第三章 判词

第四章 寒冬

第五章 钥匙

第六章 分岔

第七章 猫眼

第八章 孤岛

第九章 正名

第十章 移山

第十一章 惊雷

第十二章 风向

第十三章 悖论

第十四章 开门

第十五章 高塔

第十六章 失语

第十七章 箱子

第十八章 回眸

第十九章 末座

第二十章 减法

第二十一章 并流

第二十二章 记号

第二十三章 众目

第二十四章 代价

第二十五章 定形

第二十六章 曲线

第二十七章 三个数字

第二十八章 通吃

第二十九章 野马

第三十章 缰绳

第三十一章 褒贬

尾声 像与真

楔子 一条线

1957年，美国。康奈尔大学的实验室里，一个叫弗兰克·罗森布拉特的心理学家造出了一台机器。它的全部本事说出来几乎要让人发笑：**在一张纸上划一条线，把东西分成两堆。**就这么多。

然而当时的报纸完全不这么看。《纽约时报》的报道里，这台机器将来会走路、会说话、会繁殖、会自我复制，甚至会意识到自己的存在——而那不过是一台只会划线的机器。吹牛这件事，从1957年就已经有人干得很熟练了，**一吹就是七十年。**

罗森布拉特本人倒没有吹。**他清楚地知道自己手里这个东西能做什么，也清楚地知道它做不了什么，**他只是相信，这条路是对的。

他确实走对了。

六十多年后，那个替我们写文案、翻外语、看照片、写代码、陪人闲扯的东西，顺着血统往上数七十年代，祖宗就是这台只会划线的机器。下面这本书要讲的，就是这条线怎么长成了后来那副模样。

它**被宣判过两次死刑，被扔进过垃圾桶，被主流学界冷落了将近三十年；**它最潦倒的那段日子里，做它的人要躲躲闪闪地干活，**还得换个名字发表论文，**否则拿不到经费。

然后它赢了。**不是靠哪一个天才灵光一闪赢的，而是靠一群人前赴后继地填坑：**有人把坑填平了，有人填到一半死在了坑里，也有人认错了方向，白白搭进去二十年。

这是这个领域最奇特的一点。它不像很多科学史那样充满天才的光辉，它更像一部关于误读、运气和固执的历史——**正确的东西被埋没，错误的东西被重复，而最后改变一切的，往往是那几个没有走开的人。**

故事要从一个心理学家讲起。

第一章 初啼

罗森布拉特原本是研究知觉的心理学家，具体说，是研究人怎么“看见”东西。

他写过很厚的理论，把眼睛分成若干层，每一层各司其职：哪一层负责找边界，哪一层负责找角度，哪一层负责把碎片拼成一张完整的脸。这套书斋里的学问做得相当漂亮，在当时的心理学界颇受重视。

写完他自己却烦了。因为他发现，**他能把人怎么看东西讲得清清楚楚，却造不出一台能看东西的机器。**

于是他想了个新办法。用他自己的话来说，大意是：**我不再去描述“看见”这件事，我直接造一台会看的東西出来。**

这句话听上去平平无奇，但它其实是整个故事的起点。在此之前几百年的思路都是同一条：先弄明白这件事的道理，再把道理写成规则交给机器。罗森布拉特换了一条：**不管道理，先让它自己长出道理来。**

他做出来的东西，叫感知机。

具体做法朴素得有些过分。把一张图给它看，它把图上的每一个点换算成一个数字，于是它面前就摆了一堆数字；然后，它像拿着一把尺子一样，在这堆数字里划一条线。**线的一边判定为是，另一边判定为否。**就这么简单。

真正的巧妙之处不在尺子本身，而在于尺子是谁摆的。

尺子是它自己摆的。

流程是这样的：先给它看一批已经知道答案的例子，比如一百张图，每张图旁边都标着“这是猫”或者“这不是猫”。它先胡乱划一条线，划完之后对答案，对了不动，错了就把这条线往正确的方向挪一点。一百个例子看完，这条线挪了一百次；再来一遍，再挪一百次。几轮下来，这条线就不再是人划的了，而是它自己量出来的。

这件事在1957年是石破天惊的。因为在此之前，所有的机器都遵循同一条铁律：人把规则写进去，机器照着办。感知机不一样——**人没有写任何规则，只给了例子，规则是它自己总结出来的。**这个想法后来有了一个正式的、而且今天已经家喻户晓的名字，叫**机器学习**。而在当时，罗森布拉特管它叫“一种能自己学习的机器”。

他用硬件把这个想法做了出来：一堆电位器、光电池和马达，组装成了一个能认东西的东西。它有**四百只“眼睛”**——四百个光电池，能认简单的图形。把一张画着方块的卡片放上去，它判断这个方块在左边还是在右边，判得越来越准，尽管它从没见过任何一条关于方块的规则。

宣传机器随之开动。**军方给钱，媒体追着报**，罗森布拉特几乎一夜之间成了名人。

他自己反倒有些不安，因为他知道一件别人还不知道的事：**这条线，只能解决“能用一条线分开”的问题。而世界上大部分问题，都不是一条线能分开的。**

这个不安，日后成了他一生最大的注脚。

第二章 界碑

我们在这一章花点时间说清楚这条线的能力边界，因为后面七十年的所有故事，都是围绕着这个边界展开的。

它能干什么？**只要一个问题能被一条线分开，感知机就能解决。**判断图上的点是偏左还是偏右、邮件是垃圾还是正常、温度是偏高还是偏低——这些都是可以一刀两断的问题，刀落下去，两边分明。

那它不能干什么？**只要一个问题不能一刀切开，它就完蛋。**

最经典的例子叫异或。这个词我们在计算机课上多半听过：两个输入，相同的时候输出否，不同的时候输出是。

输入A	输入B	结果
否	否	否
是	是	否
是	否	是
否	是	是

把这四个点画在一张纸上，两个否在对角，两个是在另一条对角。画完就明白了：**无论怎么划线，都没法把两个否和两个是分开。**平着划不行，竖着划不行，斜着划也不行。这不是技巧不够的问题，是根本画不出来。

也就是说，**异或这么简单的一件事，它学不会。**

更要命的地方在于，这不是“它不够聪明”的问题，也不是“多给它一些例子就能教会”的问题。这是结构性的限制：**只要它的工作原理是“划一条线”，它就永远学不会异或。**这就好比想用一把直尺去量一个球的周长——**不是尺子不够长，而是这活儿本来就不是直尺该干的。**

罗森布拉特知道这件事。他在六十年代就公开承认过感知机的局限，与今天流传的印象相反，他从来没有把感知机吹成一个万能的东西。

但他同时也认为，这个问题是可以解决的：**如果一条线不够，那就摞两条线；两条不够，就摞三层。**底下几层各划各的线，上面一层再拿底下几层的结果去划一条新的线，堆起来之后，异或这种交叉的局自然就解得开。

这个想法是对的。不但对，而且正是七十年后那套东西的雏形。

问题在于，他提出这个想法的时候，**没有人知道怎么训练一个摞起来的東西。**

一层的東西好办：它判断错了，把那条线挪一下就是，错在哪里、该谁负责，一目了然。可摞了三层呢？最上面那层判断错了，到底该怪它自己，还是该怪底下某一层把数字报歪了？**底下那层改动一点，上面两层跟着全变，你根本无从下手。**

罗森布拉特晚年花了很长时间琢磨这个问题，留下了一些手稿，寄给同行，大部分没有回音。

他没有等到答案。**1971年，他去世了，四十三岁。**

而在他去世之前两年，有人写了一本书，直接把他这条路掐断了。

第三章 判词

1969年，一本书出版了。书名叫《感知机》，作者是两个人：马文·明斯基和西摩·派珀特。

这两个人在当时的人工智能圈子里都是响当当的人物。**明斯基尤其大牌**，他后来拿了图灵奖，被称作“人工智能之父”之一，那个年代的经费、会场和话语权，很大程度上握在他这样的人手里。

他们在这本书里做了一件事：把感知机能做什么、不能做什么，一条一条列了出来。列得极其严谨，极其客观。

然后他们指出：单层感知机连异或都学不了。并且进一步证明，就算摞成多层，当时也不存在可行的办法把多层训练出来。

这句话本身没有错。

这才是最要命的地方——**他们说的全是对的。**

但书一出来，全世界的理解变成了另外一句话：**这条路走到头了。**

这就是学术界的运行方式。**一句话说的是什么，和它被理解成什么，往往是两回事。**有人冷静地指出“你这车的最高时速只有三十公里”，这句话是事实；但传到投资人耳朵里，就变成了“这车是个废物，别投了”。然后钱就真的撤了。

明斯基其实未必想掐死这条路。他后来自己也做过一些神经网络方面的工作，态度并不激烈；但在1969年，他是那个签下判词的人，而判词的效力，不是作者能控制的。

还有一个更微妙的地方。这本书里“多层当时训不出来”的判断，**是一个当下的事实，而不是一个永久的结论。**技术会往前走，方法会被人想出来，这些后来都发生了。但在1969年，没有人会这样想。

于是接下来发生的事，我们大致都能猜到：**钱没了，人散了，这条路被扔进了垃圾桶。**

那本书出版之后，全世界的资助都从一个方向转向了另一个方向，后者的名字叫符号主义——大意是：不必让机器自己学，**我们把知识一条一条写成规则，让机器去查规则就行了。**

这条路当时看起来要靠谱得多。规则是人写的，人可以检查、可以修改、可以向资助人解释；不像感知机，划出一条线，谁也不知道它为什么这么划。于是接下来的二十年，人工智能的主流是专家系统。

后来的事实证明，这笔账算错得很彻底。而在1969年，几乎没有人怀疑过这个选择。

第四章 寒冬

先说一句公道话：**专家系统并不是没用。**

八十年代，各种专家系统确实上了不少项目。给内科医生用的诊断系统，给矿藏勘探用的分析系统，都有一批真在跑的东西，医院、矿场、大公司都肯掏钱买。它的思路也不难理解：把一位专家的经验拆成几百上千条“如果……那么……”的规则，写成数据库，机器拿到问题就一条条去匹配。比如：如果病人发烧，并且咳嗽，并且白细胞升高，那么怀疑是细菌感染。

听起来挺靠谱。

问题出在两个地方。

第一，规则写不完。要让系统处理真实世界，就得把真实世界的所有情况都写进去，而真实世界的特点是——**例外比规则多**。写“鸟会飞”，那企鹅呢？写“除了企鹅”，那受伤的鸟呢？写“除了企鹅和受伤的鸟”，那鸵鸟呢，幼鸟呢，被绑住的鸟呢？这个清单可以无限延长，而每延长一条，都要重新检查它跟前面所有规则有没有冲突。

第二，更要命：**这些系统不会自己变聪明**。一个专家系统上线的时候是什么水平，三年之后还是什么水平。它不学习。想让它进步，就得雇人重新写规则，而这三年里世界已经变了。

回头对比一下感知机：它虽然只会划一条线，但它有一个专家系统永远没有的东西——**它会自己改自己**。

这两条路的胜负，其实在起点就已经定了。只是当时谁也没看出来。

八十年代中期，专家系统的泡沫破了。项目一个接一个下马，公司一家接一家关门。这场崩盘在历史上有个专门的称呼，叫**第二次AI寒冬**——注意，是第二次，第一次就是前面那本书引发的。

那几年的气氛，可以用一句话概括：**人工智能这四个字，成了骗钱的代名词**。靠这个申请经费，评审看一眼标题就把申请书扔了。所以在很多地方，做神经网络的人会换个说法，管自己的研究叫“模式识别”或者“自适应系统”，反正不叫人工智能。

现在回头看，这段历史里最让人唏嘘的一点是：

被扔掉的这条路，其实在1986年就已经被修好了。

修好它的人，当时就坐在离那些废弃项目不远的地方。**而其中一个人，从此把同一件事讲了三十年。**

第五章 钥匙

1986年10月，《自然》杂志登了一篇论文，题目翻译过来大意是“通过反向传播误差来学习内部表示”。作者是三个人：大卫·鲁梅尔哈特、杰弗里·辛顿、罗纳德·威廉姆斯。

这篇文章解决的，正是1969年那本书里判了死刑的那一条：多层怎么训。

它给出的答案，**今天有一个如雷贯耳的名字——反向传播。**

我们用一条流水线来理解它。假设你办了个厂，流水线上有十道工序，产品从第一道进去，从第十道出来，最后检验发现是次品。问题来了：该怪谁？怪第十道，还是第九道，还是第一道把料给错了？

如果只有十道工序，你可以一道一道地试：把某一道稍微调一下，看看成品率会不会变好。这办法可行，但你得试很多很多次；工序一多，试试看这条路就废了。

反向传播说：不用试，我算得出来。

它的做法是**从最后的错误开始，往回退**：先看最后一站的错有多大；再弄清这个错里，最后一站自己占多少、前面九道工序占多少；然后把前面那部分责任继续往前分摊。一处分摊成两份，两份分摊成四份，一直分到第一道工序。分完之后，每一道工序都知道：在这次错误里我占几分，我该怎么调。

一次回溯，十道工序同时找到方向。

这就是反向传播的全部意思。数学上它用的是微积分里一条早就存在的老规矩——链式法则；但把它用在多层网络上，把“责任”这个概念变成可以逐层计算的东西，是这三个人干成的。

这里有一个细节值得说。这个想法其实1974年就有人在博士论文里写过了，作者叫保罗·沃博斯。**只是它没有激起任何水花。**

为什么没有人注意？因为1974年，神经网络这个领域已经快没人了。**一个东西在没人关心的时候发布，等于没有发布。**这也是科研界的一条规律：**同样一个想法，什么时候说出来，往往比说了什么更重要。**

回到1986年。

这篇文章一出来，异或那个死局立刻解开了。鲁梅尔哈特他们当场演示：一个三层的网络，把感知机怎么都学不会的异或问题干净利落地解决了。现场的反应，先是有人不信，然后是自己动手试，然后是一片哗然。

因为它证明的并不只是一个玩具题。它意味着：**只要层数够，一个网络可以表达任何复杂的函数。**

限制解除了。

很多人以为，接下来就该是神经网络横扫天下：1986年发现方法，1987年大批跟进，1990年代遍地开花。

不是的。

因为这三个人走出门口，立刻撞上了第二堵墙。

第六章 分岔

第二堵墙，我们今天叫梯度消失。

先说它长什么样。反向传播是把错误从出口往回分摊，问题在于，每往回退一层，那份错误就要乘一个系数，而这个系数的值通常小于1。乘一次是0.7，乘两次是0.49，乘三次是0.34，**乘十次剩下不到3%**。退到浅层的时候，那份错误已经小到可以忽略不计了。

浅层什么感觉都没有。它不知道自己错了，所以它也就不改。

结果就是：搭一个十层的网络，实际能训动的只有上面三层，底下七层等于白搭，甚至还是负担。

这个现象有一个很传神的比喻，叫**传话游戏**。十个人站成一排，第一个人说一句话，一个一个往后传，传到第十个人那里，早已面目全非。**而神经网络遇到的还不是面目全非，是彻底没了。**

辛顿后来说，那几年他常做一个梦：梦见一条极长的走廊，他提着灯往里走，走到一半灯就灭了，再往前走，脚下就没有路了。

他们试过各种办法，都不好使。把系数调大，训练就炸；把层数减少，又回到“不深不管用”的老路上。

就在这个时候，学术界补了一刀。

1989年，有几个搞数学的人证明了一条定理，内容非常简洁，一句话：只要你要拟合的东西是连续变化的，一个隐层就足够逼近它，要多少精度有多少精度。

翻译成成人话就是：**一层就够，不用堆。**

这就是著名的通用逼近定理。

必须强调：这个定理是真的。它被严格证明了，没有错。

但它被理解成了另外一句话：既然一层就够，何必费劲堆深？

这个误读，把深度网络又按回去了二十年。

问题出在“逼近”这两个字上。定理说的是能表示——理论上存在一个单层网络能表示你要的那个函数；**它没有说能学到，更没有说需要多少参数。**

这才是真正埋着坑的地方。定理在数学上完全正确，因为**数学上的“存在”是个很宽松的概念**：只要宇宙里存在这么一组参数，定理就成立。至于这组参数要用多少个神经元、要训多久、能不能训出来，定理一句都没有承诺。

实际情况是：要让一个浅层网络表示深度网络轻易就能表示的某个东西，需要的神经元数量会呈指数级地涨上去。换句话讲，一个十层网络用几千个参数就能干好的活，一个单隐层网络可能要用几十亿个神经元来表示——理论上能表示，实际上根本训不出来，训出来也跑不动。

一层就够，是真的；一层就够，同时也贵到不可行。

辛顿当时就说过一句很到位的话：**那句定理说的是能不能表示，不是能不能学到。**

这话他说了不止一次。但当时没有人听。

因为大家正忙着为别的事情兴奋。那几年有一批方法横空出世：支持向量机、随机森林，以及各种精巧的统计学习方法。它们有一堆让人舒服的优点——数学漂亮、效果稳定、可以解释、在小数据集上表现极好。相比之下，神经网络像个黑箱，还训不动，还容易过拟合。

于是九十年代的风向彻底转了，神经网络被挤到了角落里。那二十年里，学术界的主流共识可以概括成一句话：**智能的关键在特征，而特征得靠人设计。**

这话在当时是不折不扣的常识。要认字，就人工设计笔画的统计特征；要认脸，就人工设计眼距鼻距的比例；要分辨声音，就人工造频谱的形状。谁把特征设计得越巧妙，谁的本事越大。这个行当养出了一大批很受尊敬的人，他们不叫神经网络专家，叫特征工程师。

而那些深网络的旧论文，被压在了箱底。

辛顿在这个阶段做的事情，是继续讲他那句话。**从青年讲到中年，从中年开始头发变白**，讲的始终是一件事：深是有用的，而且深的好处是浅的东西代替不了的。有人当面拿那条定理反驳他，他就回那句“能不能表示和能不能学到，是两回事”。

没有人听。

但有一件事在那个年代悄悄发生了，而且后来被证明比这一切都重要：换了一批人。新一代的研究者从不同的地方、带着不同的思路，慢慢摸到了几条新路。

其中一条路的起点，是一位年轻人在一只猫身上受到的启发。

第七章 猫眼

九十年代，神经网络在学术界基本没戏了。但在工业界的一个角落里，有个法国年轻人正在做一件挺不一样的事。他叫杨立昆（Yann LeCun），后来被称作“卷积网络之父”。

他早年也是个特征工程师，而且是很强的那种。强到什么程度？强到他自己烦了。他说过一句话，大意是：**我每设计一个特征，就等于多写了一条规矩，这世界哪有这么多规矩。**

于是他转向了另一个方向：能不能让网络自己学会该看什么？

启发他的不是别的，是一只猫。

1962年，两位神经科学家**胡贝尔和维泽尔**做了一个很有名的实验。他们把电极插进猫的视觉皮层，给猫看各种图案，记录神经元在什么时候放电。结果发现了一件很有意思的事：**视觉皮层里的神经元并不是对整张图做出反应，而是对图上的一个小区域做出反应。**有的神经元只看一小块，专门对某个方向的线条兴奋——把线条的角度转一下，它就不理你了；还有的神经元范围大一点，对“某个方向有线条”这件事有反应，但不太挑具体位置。

这个发现后来拿了诺贝尔奖。它的意思是：**生物视觉不是整张图一起看的，而是分层处理的——先看小范围的边缘和方向，往上再拼成更大的结构。**

杨立昆盯着这个发现，想出了两件事。

第一件事，是局部。以前的全连接网络是这样的：上层每一个神经元，都要跟底下所有的像素各连一根线。一张一千乘一千的图就是一百万个像素，上面一个神经元要连一百万根线，一层下来就是上亿个参数。这不是在算，这是在烧。卷积说：不必，上层每个神经元只看一小块，比如三乘三的九宫格，九个数字进去，一个数字出来。

第二件事更关键，叫做共享。看九宫格的那套配方——也就是那九个权重，在同一张图上从头到尾都不换。左上角用它，右下角也用它，中间也用它。为什么可以这样？**因为一只猫的耳朵，不管长在画面左边还是右边，都还是耳朵；**同一个东西出现在画面上任何地方，都应该用同一个办法认出来。

这件事，不用教给机器，杨立昆直接把它写进了网络的结构里。

两条一合，效果是惊人的。**原本一层要上亿个参数，现在只要九个。**省下来的不是一点，是几万倍。

更妙的是，那九个数字究竟是多少，不需要人来设计。给它看几千张有耳朵的图，**它就自己长出一套认耳朵的配方；**给它看几千张有轮子的图，它就长出一套认轮子的配方。

这里有一个分寸特别值得说清楚，因为它是整个深度学习最重要的一步。九十年代那些特征工程师做的事是：人去想该看什么——人设计出“眼距除以鼻距”这样的特征，交给机器使用。而卷积做的事是：

人只规定在哪儿看、开几套配方，看什么由机器自己长。

人管的事，从设计内容退到了设计形式。

这一步退得很小，却退出了一个新天地。

顺带说一个当时被人忽略的好处，它后来变得极其重要：因为配方是共享的，所以不管猫出现在画面左边还是右边，算出来的结果都一样。这个性质叫做平移不变性。对图像来说它极其重要，而工程师们根本不必专门去实现它——**它是从结构里自然长出来的。**

第八章 孤岛

光有想法不行，还得看能不能干活。

1989年，杨立昆在贝尔实验室用反向传播训练了一个卷积网络，让它识别手写的邮政编码，效果不错。此后他做了很多年的迭代，到1998年，做出了一个五层的版本，叫 **LeNet-5**。

五层。没有看错，就五层。

这个五层的小网络能认手写数字：写一个“7”上去，它告诉你这是7。它在当时的测试集上错误率约为百分之一。

现在说重点：**它是第一个真正赚到钱的东西。**

美国有家做自动柜员机的公司叫NCR，把这套东西装到了读银行支票的机器上。手写金额、手写账号，以前只能靠人一张张看，八小时下来眼睛都花了。这套东西上线之后，**据说一度全美国有百分之十左右的支票是它读的。**

在一个所有人都说“神经网络没戏”的年代，它每天在几万台机器上，安静地读着几百万张支票。

这是那段黑暗时期里，**唯一一个真正在跑、真正在赚钱、真正有人付钱的神经网络。**

不过它有一个天花板，而且卡了很多年：**它只认小图。**

LeNet-5处理的图像是二十八乘二十八的——横竖各二十八个点，黑白的，干净的，只有笔画没有背景。把一张真实照片拍给它，它就傻了。

原因有两条，两条都很硬。一条是视野太小：五层网络，每一层只看得到周围几十个像素，而一张真实照片里一只猫可能占六百个像素宽，它一次只能看到猫的一个斑点，根本拼不出“猫”这个概念。**另一条才是真正卡死它的——没数据。**

要让网络学会认真实世界里那些歪七扭八、明暗不均、角度千奇百怪的东西，得给它看几十万、几百万张标好了答案的照片。而当时全世界的学术界加起来，能凑出来的标注图像也就是几万张。

还是个位数万的量级。

杨立昆自己很清楚这一点。他后来反复说过一句话：**我这套东西，不缺想法，缺世面。**

“缺世面”这三个字，是接下来十年整个领域的主题。

九十年代末到两千零几年，神经网络面临的局面是这样的：方法有了，反向传播；结构有了，卷积；就差两样东西——数据和算力。**而这两样，都不是学术圈自己能变出来的。**

它们得等。等硬件便宜下来，等互联网把海量的数据摆到桌面上。

而在等的这段时间里，有人做了一件看起来完全是歪门邪道的事。他没有去改进方法，而是换了个说法。

第九章 正名

2006年，辛顿五十九岁。这一年他发了几篇论文，**提出了一个方法叫深度信念网络**，核心技术叫逐层预训练。

先说这个方法讲了什么。

前面说过，反向传播往回退的时候，力会一层层衰减，退到浅层就没了。辛顿这一次不去跟它硬碰，而是绕开：不一次把整个网络教完，而是一层一层地养。

具体做法是：先单独训练最底下那一层，只让这一层看数据，不管别的，养到它自己对数据有一套说法——哪种输入常常一起出现，哪种输入互相排斥——这一层就算有样子了。然后把它冻住，在它上面再架一层，拿它吐出来的东西当输入接着养；养好了再冻住，再上一层。一层一层往上搭，搭完之后再解开所有冻结，从头到尾整体调一遍。

这里最关键的是“冻住”这两个字。底下那几层被冻住的时候，反向传播根本传不到它们那里去，所以也就不存在“力衰减到没有”这个问题——反正不上传。而等到它们被解冻、参与整体调整的时候，它们已经不是随便乱摆的一层了，它们已经有了一套自己的说法，这时候反向传播给的那点微弱的力量，它们接得住，也改得动。

先立后调，中间冻住。这就是逐层预训练。

它有用吗？

有点用，但不多。老实说，这个方法在实践中的效果没有达到当时的预期。它让深层网络能训了，但训得又慢又不稳，跟当时那些靠人工设计特征的方法比，也并没强出多少。

辛顿自己心里清楚。他的这个方法，真正的作用是另一个，而且非常关键：**他让“深”重新变成了一个可以摆到桌面上谈的话题。**

而在做这件事的同时，他顺手做了一件更聪明的事：换了个名字。

他不再管这个东西叫多层神经网络，**他管它叫深度学习。**

这话听起来有点像玩笑，换个名字算什么本事？但里面的门道值得说清楚。

在2006年的学术界，“多层神经网络”这五个字是个不吉利的词。提起它，人们想到的是梯度消失、训练不动、九十年代那批失败的项目、被扔进垃圾桶的那条路，评审一听见这个名字脸色就变了。而“深度学习”这四个字，一下子把话题偷换了：它不谈这套东西到底行不行，它谈我们要做多深。

同一件事，换个名字，就从一笔旧账变成了一道新题。

这个操作在历史上很常见。一个被证伪过的方向如果想要复活，第一步往往不是做出新成果，而是先换一个不会让人产生条件反射的名字。

这就是2006年真正发生的事。

顺便说一句，辛顿这一辈子有个特点：他被驳倒过很多次，但每一次都还在讲。从1986年讲反向传播，到2006年讲深，中间隔了二十年，二十年里他讲了无数场没有几个人听的报告。

他不是最有天赋的那个人，也不是数学最好的那个人。**他是那个没有走的人。**

而2006年之后的三年里，有人把最后一块拼图放上了桌。放拼图的这个人，做了一件跟当时所有人都不一样的事。

第十章 移山

她叫李飞飞。2006年前后，她在普林斯顿当助理教授。

那时候整个领域的聪明人都在干什么？都在改进算法。怎么把网络结构设计得更精妙，怎么把特征的表达做得更紧凑，怎么让模型在小数据集上表现更好一点。**所有人都在磨刀。**

李飞飞做了一件在旁人看来很笨的事：**她去数了数，世界上到底有多少张标好的图。**

数完她心里发凉。所有公开的数据集加起来也就是几万张，还要分成几十个类别，每个类别只剩下几百张。几百张图，要教会一台机器认得一样东西。她说，**这就好比只给一个孩子看过三张椅子，就要他认遍天下所有的椅子。**

于是她决定不磨刀了，**她去找肉。**

这个决定的难度不在想法上，在执行上。她要找的是几百万、上千万张图片，每一张都要有人看过，并且写上“这张图里是什么”。两千零几年没有现成的东西，她得自己组织人手，一张一张地标。

她做了一件在当时很有创造性的事：用众包。那时候亚马逊有个服务叫土耳其机器人（Mechanical Turk），可以把一件小活拆成无数份，发给世界各地的普通人做，做完给一点小钱。李飞飞就靠这个雇了几万人，一张一张地看图、打标签。

这个活的规模有多大？最后做出来的东西叫 ImageNet：**一千四百万张图，两万多个类别，每一张都有人工标注。**

一千四百万张。而在此之前，整个领域的标准是几万张。

这不是量变了，这是换了一个世界。

ImageNet 在2009年发布。**反应是什么？嘲笑。**

当时的主流评价大概是这个意思：堆数据算什么本事，谁不会堆？你堆了几千万张图，方法还是老方法，有什么用？还有人说，这个数据集太大了，根本训不动，没人用得上。

李飞飞一句也没有辩。她只说了一句话，后来被反复引用：**这个数据集不归我，也不归普林斯顿，全世界谁要，谁拿去用。**

她真的把门开得大大的。

这句话在当年被理解为一种示弱——我做的东西没什么了不起，大家随使用吧。但几年之后回头看，它是那个时代对整个领域最大的一次贡献。因为接下来发生的事情是这样的：ImageNet 发布后三年，有一个方法在这个数据集上炸了。而那个方法，如果没有这一千四百万张图，连门都进不去。

与此同时，还有第二件事在悄悄发生，只是它当时完全没有进入学术圈的视野。

2000年代中期开始，显卡的发展进入了一个疯狂的阶段。而显卡这东西，本来是给游戏玩家造出来的：它要干的事是每秒画几千万个像素，每个像素的颜色都要算。这种活的性质是——大量的、互不干扰的、可以同时进行的简单计算。

这跟神经网络的运算性质一模一样。

这是后来那件大事的前置条件。而在当时，没有人把这两件事联系起来。

两年之后，有人把它们拼到了一起。

第十一章 惊雷

从2010年开始，有一场比赛每年办一次。比赛内容很简单：给你一百五十万张带标签的图，涵盖一千个类别，让你训一个模型，然后拿十万张没见过的图考你，看你能认对多少。这场比赛叫 ILSVRC，用的是 ImageNet 的数据。

2010年，冠军是传统方法，错误率28%。2011年，冠军还是传统方法，错误率26%。27%左右这个数字已经卡了很久，很难再降。

2012年，来了三个人：亚历克斯·克里哲夫斯基、伊利亚·苏茨克维、杰弗里·辛顿。前两个是辛顿的学生。

他们交上去的东西叫 AlexNet。

先说他们用了什么，一共四样。

第一样，卷积网络。这个前面讲过，杨立昆十几年前就有了。

第二样，ReLU。这是整个故事里最便宜也最漂亮的一个改动，必须专门讲。以前网络的每一个神经元出口都装着一个“压缩阀”，叫 sigmoid 或者 tanh，作用是把输出压到一个固定范围里，听着挺合理。但它有一个大毛病：两头是平的。输入很大的时候输出几乎是1，再怎么加大也还是1；输入很负的时候输出几乎是0，再怎么减也还是0。而平的地方斜率就是零，反向传播靠的恰恰是斜率——**斜率是零，梯度就是零，这一层就收不到任何信号了。**网络里有的是神经元会被推到两头，一旦推到那儿，它就死了，再也学不动。

ReLU 把这件事解决得粗暴又漂亮：**正数原样照走，负数直接归零。完了，就这么简单。**正数那一头不压缩、不变平，梯度原原本本往回走；负数那一头虽然被削平了，但因为规则简单，算起来还快了好几倍。

这个改动在当时看起来非常粗糙——一个几乎不“非线性”的东西，怎么能当激活函数用？结果它比那些精雕细琢的阀门好用得多。有时候历史就是这样，被最认真设计出来的东西挡住了路，而被最漫不经心的一笔放了过去。

第三样，Dropout。训练的时候每一轮随机把一半的神经元关掉。为什么这么干？防止网络偷懒。如果某个神经元特别能干，其他神经元就会靠它，整个网络退化成几个神经元的表演，一遇到没见过的数据就崩。随机关掉一半，每个神经元都得学会独立干活，这相当于每次都在训练一个不同的子网络，而测试的时候把它们全部打开，效果是一群各有本事的模型在投票。

第四样，数据增强。把 ImageNet 的图翻过来、剪一块、调一下明暗色调。一千四百万张，折腾一圈变成几千万张，而且张张不完全一样。这个操作看起来像作弊，实际很聪明：一张图翻个面它还是那张

图，“翻转之后答案不变”这件事等于白送的正确答案。训练的时候告诉网络——这两个看起来不一样，但答案是一样的，这本身就是宝贵的信息。

再说他们烧了什么。

两块 GTX 580 显卡。这是当时市面上算挺好的游戏显卡，一块3GB显存。**3GB显存放不下他们的网络——一共六千万个参数。**

怎么办？他们把网络从中间劈成两半，一半放一块卡上，两边在中间接一下。

这就是那个年代的做法。今天动辄几千亿参数的模型，当年连一个六千万参数的网络都装不进一块显卡。

结果出来那天：**Top-5 错误率15.3%**。第二名是26.2%，用的是传统方法。

这个差距有多大，需要稍微解释一下。在那之前，大家比拼的是小数点后面第二位；这一回，一口气削掉了将近一半，而且直接甩开第二名十个以上的百分点。

这不是赢了，这是换了一个物种。

据说结果公布的时候，台下第一反应是沉默，然后有人开始算自己手里的项目还有没有必要继续。

有一件事值得一提：辛顿那年六十五岁，坐在会场的后排。

他等这一天，等了二十六年。

第十二章 风向

2012年之后，这个领域发生了一次非常彻底的换血。

第一件事，所有人都转了。2013年，全世界做计算机视觉的实验室**几乎全部转向了深度学习**。前一年还在讲特征工程的课题组，第二年就都在训卷积网络；论文标题里带“deep”的，数量一夜之间翻了好几倍。

第二件事，简历市场崩了一次。那些做了十年特征设计的研究生和工程师，忽然发现自己引以为傲的手艺没人要了。

这不是开玩笑。一个博士生花了四年时间，把某个特定领域的特征设计得炉火纯青，毕业的时候发现所有人都在用同一套网络结构，直接喂数据就行，根本不需要他那些精巧的设计。

他的四年，白念了。

这件事在后面还会反复发生。**每一次技术换代，都有一批人的技能被清零。**这是这个领域的规律，残酷，但真实。

第三件事，大家得出了一个后来被证明是错的结论：**既然深有用，那就往深处堆。**

2014年有两篇论文几乎同时出来。一篇叫 VGG，作者的想法极其朴素：就用统一的小卷积核，一层一层往上堆，堆到十九层，结果比 AlexNet 好。另一篇叫 GoogLeNet，思路活一些：在同一层里并排放好几组不同大小的卷积，同时看不同尺度的东西，另外还在中间插了一些“窄门”用来降维，最后堆到二十二层。

两家加起来，结论是一样的：比八层强。

于是整个领域进入了盖楼竞赛。二十五层、三十层、四十层，论文里的层数一本比一本厚。

然后所有人一起撞上了墙。

而且这堵墙很怪。它不是“练不出来”或者“效果不好”这么简单，它违背了一个几乎所有人都觉得理所当然的直觉。

第十三章 悖论

2014年底到2015年初，有人做了个实验，结果所有人都不信。

他们训了两个网络，一个二十层，一个五十六层。训练方法、数据、超参数全部一样，唯一的区别就是层数。

按常理，五十六层的表现至少不该差于二十层——就算多出来的三十六层什么都不干，学成“原样照抄”，那也不该更差。结果却是：**五十六层的比二十层差。**而且差得还不是一点。

一开始大家以为是过拟合。过拟合是很好办的：减少参数量、加 Dropout、加正则化，处理一下就好了。于是他们把这些都做了。

还是差。

这下没法用“背题”糊弄过去了。于是有人去看了一个平常不太看的数字：训练误差。

一看，全明白了。五十六层网络在**训练集上的误差，也比二十层高。**

这就是那件怪事。这里需要把两个概念分清楚：过拟合是练的时候很好、考的时候很差，是背题背多了，题目稍微换个说法就不会；而眼下这个毛病是——**连练的时候都不好**，题目都见过，还是做不对。

你给它更多的层、更强的表达能力、更多的计算量，它在自己家里反而更笨了。

当时这事儿被叫做退化问题。

从直觉上，它完全说不通。因为五十六层网络的表达能力是包含二十层网络的：只要把前面二十层学成二十层网络的样子，剩下三十六层学成“原样传下去”，也就是恒等映射，那它的表现就应该和二十层

一模一样。

这个解一定存在。

那为什么找不到？

答案是：存在和找得到，是两回事。

这句话在前面听过一次了——1989年那句“一层可拟”的定理，坑的就是这里。这一次，坑的是另一个推论：更深的网络里一定包含浅网络的解，所以深网络不会更差。

能表示，不等于能找到。

更深的空间里，好解是存在的，**但优化器找不到它**。而且层数越多，找路这件事越难。可以这样理解：一个更宽广的房间，理论上更好布置，但你在黑暗里摸，**房间越大越容易迷路**。

那两年的解决方案，都往同一个方向使劲：把信号往上挤。批量归一化是其中最著名的一个，它的作用是把每一层输出的分布强行拉回一个稳定的范围，别让数值忽大忽小。这个改动确实管用，能让网络训到二三十层。

但它的本质还是“挤”——挤得越用力，能多上几层；可挤到某个地方，还是要断。

没有人问过另一个方向的问题：为什么要让信号一层层挤过去？

问这个问题的，是一个三十岁出头的中国年轻人。他给出的答案，简单到让人怀疑。

第十四章 开门

2015年，何恺明三十一岁，在微软亚洲研究院。

他做的那个改动，如果只看公式，可能会让人觉得“就这？”

原来的规则是这样的：每一层都要把上来的东西变成一个全新的东西。上一层给你一堆数字，你负责把它算成另一堆数字交上去，每一层都是链条里不可或缺的一环。

何恺明改成了这样：**每一层只需要学“该添多少”**。上一层给你一堆数字，你照旧去算，但算出来的结果不当成交上去的东西，只当是增补；真正交上去的是——**原来那堆数字，加上你算出来的增补**。

就这么一句话。

这个改动带出来两样东西，一样比一样重要。

第一样是那条捷径。既然交上去的是“原样加增补”，那就**必然存在一条路从最底下直通到最上面**：原样那一份，谁也没有动过它，它自己就上去了。这条路后来被叫做恒等捷径。

它的作用是什么？**反向传播终于有直路可走了。**前面说过，梯度往回退，每退一层要乘一个小于1的系数，乘着乘着就没了；现在有了一条捷径，那份梯度可以从捷径上原样退回，一点不打折。想退到多浅就退到多浅，退到哪儿，那份信号都还是完整的。

何恺明打过一个比方，说得特别到位：**门如果是关着的，你就得把风拼命往里挤，挤到后面就散了；门如果是开着的，风自己就走过去了。**

第二样比捷径更深。它在每一层的最坏情况上，划了一道底线。

让一层学“变成一个全新的东西”，其实是件很难的事——你不知道该学成什么样。而让它学“添多少”就容易多了，因为**最容易的一种“添多少”，就是添零**。一层如果学不出任何有用的东西，它只需要把增补学成零，交上去的还是原样。

这一层等于没白搭，也没有帮倒忙。

这就给了所有深网络一件保底的东西：每往上加一层，最坏的结果是原地不动，不会更差。

“最坏是原地不动”，这七个字才是真正把深度放开的钥匙。因为在此之前，每加一层都是一次赌博：可能变好，也可能变差，而且变差的概率不小。有了这个保底，加层这件事就从“赌一把”变成了“最差也不亏”。

话说回来，这个想法在当时并不是全新的。“把输入直接加到输出上”这种做法，早年间在某些网络里出现过。但没有人系统地证明它能让深度网络往上堆几百层，也没有人想到它是解开退化问题的那把钥匙。

想法早就有，是把想法推到极致的那个人赢了。

第十五章 高塔

实验做出来之后，第一件事是把层数往上堆。既然最坏是原地不动，那就堆吧。

于是他们一层开一条捷径，**一口气盖到了一百五十二层。**

一百五十二层。

为什么是152？因为再往上加，实验成本太高了，而且当时的收益曲线已经开始变平。不是因为堆不动了，是因为没有这个必要。

这个一百五十二层的网络用在 ImageNet 上，**Top-5 错误率是3.57%**。而人类在这项任务上的表现大约是5%。

机器第一次在这项比赛里比人还准。

不过更重要的不是这个数字，是这个数字背后的那件事：**深度没有上限了**。在此之后，一百层、两百层、上千层，往论文里写也没什么奇怪的。

而残差连接这个东西，很快就不再是某一篇论文的卖点——它被写进了几乎所有主流网络的结构里，**成了标配**。我们今天听到的任何一个现代网络，从卷积到 Transformer，里面都有它。

顺便说一句，它可能是整个深度学习里最便宜也最深刻的一个改动。说它便宜，是因为它只改了一行公式；说它深刻，是因为它解决的不是某个具体任务的精度问题，而是一个根本性的问题——如果我多加一层，会不会更差？

在此之前，这个问题没有稳妥的答案，所以大家不敢往深里堆。在此之后，这个问题有了确定的答案：**不会更差**。

一门技术能不能长成大东西，往往就取决于这种看起来无聊的问题有没有被解决。不是“能不能做到最好”，而是“最坏会烂到什么程度”。这就是学术和工程的区别：**学术关心上限，工程关心下限**。

残差连接和批量归一化这两样东西，一个管梯度走得到，一个管数值稳得住。两样一配齐，深网络才算真正变成了一件随手可用的工具。

于是2015年之后的那几年，是这个领域非常好的一段时光。所有人的共识是：网络结构这块，剩下的事就是精细调整了，真正的增长点在数据和算力。

这个共识大体上是对的。

但他们忘了一件事：机器到目前为止，只会看图。

而语言这件事，比看图麻烦得多。

第十六章 失语

前面这么多年讲的都是图。

图有一个极大的好处：一次全看得见，位置是死的。一张二百二十四乘二百二十四的图，横竖都有，哪个像素挨着哪个像素永远不变，所以可以一次性全部喂进去，也可以像卷积那样，让同一个探测器在图上各处扫。

语言不是这样。

语言有先后，而且先后极其重要。我们可以做个小小的实验：把“狗咬人”三个字调个位置，变成“人咬狗”，只是两个字换了个次序，一个说狗疯了，一个说人疯了。所以**不能把一句话拆成一堆字摊在桌上**——摊开了，意思就没了。

要听懂一句话，必须按顺序听。

最早解决这个问题的办法，叫循环神经网络（RNN）。它的思路很朴素：看完一个字，把此刻心里的结论含在状态里，带着这个状态去看下一个字。一个一个字往前走，状态一直被带着，走到句尾，这个状态里就装着整句话的意思了。

可以把它想象成一个人边听边记小本本：每听一个字，就在小本本上更新一下，听完，小本本上就是整句话的大意。

这个设计的好处很明确：句子多长都行。五个字能走，五百个字也能走，因为它一次只处理一个。

坏处也很明确，而且两个都很硬。

第一个坏处是慢。因为它必须一个字接一个字地走，前一步不算完，后一步开不了口。这意味着它没法并行——手边有一千块显卡，它也只能用一块，剩下的九百九十九块在那儿干看着。

前面刚说过，ImageNet 那个时代最大的瓶颈是算力；结果语言这边倒好，算力再多也没用。**不是算力不够，是这份工作本身没法交给很多人同时干。**

第二个坏处是忘。状态一路往下带，带得越久越稀。一句话的后半截跟前面隔了三十个字，前面那个字的意思传到后面，已经淡得几乎看不见了。**说话的人明明刚讲过，听话的已经忘了。**

这个毛病跟前面说的梯度消失是同一个道理的另一表现：信息在传递过程中衰减。

1997年，有人想了个办法，叫 LSTM。它的思路是给这个“小本本”加几道闸门：一道门管“现在这个字该记进去多少”，一道门管“之前记的东西该扔掉多少”，一道门管“此刻该把多少拿出来用”。**这三道门都是学出来的，不是人硬定的，网络自己决定什么时候记、什么时候忘、什么时候说。**

LSTM 很管用。管用到什么程度？从1997年一直到2015年前后，将近二十年，所有跟语言有关的模型，底下用的都是它。

但要注意：**它缓解了忘，并没有解决慢。**一个字一个字走这个根本约束，一道闸门也没有动过它。

所以2015年前后，语言这个领域的处境是这样的：图形那边，残差连接已经把深度放开了，网络越堆越深，一路狂飙；语言这边，还在一个字一个字地啃。

而接下来那个突破点的出现有点出人意料——**它一开始，只是给 LSTM 打的一个补丁。**

第十七章 箱子

2014年，有几个人想出了个新框架，叫 **Seq2Seq**，序列到序列。主事的是伊利亚·苏茨克维——对，就是 AlexNet 那三个人里的一个，他从图像那边转过来了。

这个框架的结构，可以想象成两座塔。

左边一座塔管读。一整句话，无论多长，从塔底一个字一个字喂进去，沿着 RNN 一路往上走，走到塔尖的时候，所有字的意思被收拢成一样东西。这样东西是一个固定长度的向量，就是一串数字，长度是死的，比如一千个数。

右边一座塔管写。塔尖拿到这串数字，从顶上开始一个字一个字往下吐，吐出一个字，就把它当成下一步的引子，接着吐下一个。

比如要把英文译成中文：左塔把整句英文读完，压成那串数字；右塔拿着这串数字，一个一个中文字往外写。

这套东西出来的时候效果惊艳。**它头一回让读和写分开办事**，两边各走各的塔，那两年的机器翻译比赛上，用这套结构的模型几乎全胜。

但它有一个死结，就藏在“固定长度”这四个字里。

那串数字的长度是固定的。一句十二个字的英文压出来是这样一串，一千个数；一句六十个字的英文压出来还是这样一串，还是一千个数。

这就是在把一整间屋子塞进一个箱子。屋子小的时候塞得进，屋子一大，就非得丢东西不可。

丢什么？**丢掉的往往是最早进去的那部分——句子的开头。**

于是就有了那个很典型的现象：短句译得好好的，句子一长，越往后越离谱，因为解码那边写着写着，把开头忘干净了。

有人打过一个比方，我觉得比“箱子”更贴切：这就像背着一袋东西过独木桥，桥越走越窄，袋子越背越沉，走到一半，你就开始把开头装进去的东西一件件往水里丢。

这个死结在2014年非常刺眼，因为它出在最不该出问题的地方——翻译这活儿，最需要信息完整。而且它还有一个更麻烦的性质：**它不是偶尔出错，它是句子越长错得越稳定地多**，只要句子够长，就一定会崩。

当时大家都清楚这是个瓶颈，也都在想怎么解决。有人试着把那个箱子做大一点，把固定长度从一千改成两千、四千。

没用。因为句子的长度是没有上限的，箱子再大也追不上；而且箱子一大，解码那边要处理的东西也变大，速度还慢了。

所以这不是把箱子做大能解决的问题。需要的是另一个思路：**干脆别装箱子了。**

一年之后，有人想到了。

第十八章 回眸

2015年，一位叫德米特里·巴达诺的年轻人和他的同事们，在 Seq2Seq 上动了一刀。

他动的地方非常聪明：**他不去改那个箱子，他去改右塔。**

原来右塔在写每一个字的时候，能看的只有箱子里那串数字。巴达诺说：何必只看箱子？左塔那边还留着整句原文呢，一个字都没丢，**为什么不让右塔回头去看？**

于是他做了一件在当时相当大胆的事。他给右塔开了一道口子：每写一个字之前，先做一件事——把当前要写的这个东西，拿去和原句里的每一个词对一遍。

对一遍算出什么？算出一串数。这串数说明一件事：此刻要写的这个字，跟原句里的哪些词关系最大。然后照着这串数，把原句里那些词各取一些，掺在一起，凑成一样新东西，跟箱子里那串数字一起交给右塔。

这个东西后来被叫做注意力。说白了就是：回头看一眼。

它的巧妙之处有两点，都要说清楚，因为这两点后面会很关键。

第一点，它是按当下所需算的，不是死规矩。翻译到中文的“他”字时，回头一看会算出来：原句里那个第三人称代词最要紧，它后面那个动词次之，其余的几乎不用看。译到下一个动词时，这串数又变了——这回最要紧的变成了另一个词。

同一句原文，每次回头看重点都不一样。这不是把原句原封不动搬过来，这是按需取用：写“他”的时候看代词，写动作的时候看动词。

第二点更根本：**它看的是原句，不是压缩过的东西。**原句六十二个词，全都还在那儿，一个都没丢。回头的时候是在里面挑，挑多挑少全凭当下需要。

它没有那个装不下的箱子。

这一刀下去，长句翻译的毛病立刻好转。原来句子一长就散架，现在六十个词的长句译出来也还立得住。

顺便说一个当时大家没太在意的好处：**注意力是看得见的。**因为那串权重是明确的数字，可以把它画出来——译出某个词的时候，模型回头看了原句的哪几个词、看了多重，一目了然。这在深度学习这个黑箱领域里是件很稀罕的事。很多年后，人们还在用这个特性去研究模型到底在想什么。

但有意思的是，**这一年没有人觉得注意力是个主角。**

第十九章 末座

巴达诺那篇论文的标题，大意是“联合学习对齐与翻译的神经网络”。

“注意力”这个词，根本没有出现在标题里。

这不是他谦虚，这是当时整个领域的共识。那几年，一说到处理语言，所有人认定的核心都是那个“环”——就是 RNN 那条一个字一个字走的链路。**环是主干，注意力是挂在旁边的零件。**

挂它是为了什么？是为了补环记不住的毛病。环走远了会忘，那就让它回头看一眼原文，这就够了。那两年的论文里，标准结构是这样的：底下是一条 RNN 链路，链路上挂几处注意力，顶上再压两座塔。把这种模型画成图，看起来是一长条弯曲的、打着圈的线，旁边挂着几个小小的东西。

圈是主，那些小东西是辅。

当时有一句话很流行，大意是：**环是根本，注意力是末节；环不能废，注意力不能没有。**这话听着四平八稳，其实把主次定死了。

而且，就连巴达诺自己也没有想到后面的事。

有人问过他一个问题，这个问题后来被反复引用：**如果不用那个环，只留注意力，行不行？**

他想了想说：不行。因为如果只看注意力、不看环，那每写一个字，都要把原句从头看到尾。原句有六十个词就是看六十次，写六十个字就是三千六百次。**字一多就看不过来了。**

这句话在当年是一句劝退的话。它说的是注意力这个东西的代价：每写一个字，要把原句完整看一遍；**原句有 n 个字，那就是 n 次，写 n 个字，就是 n^2 次。**

这个代价在当时看是致命的。因为 RNN 有一个好处，就是看一遍就够了——它一边走一边把信息收进状态里，不必反复回头看。

但这句话说出一个精确的事实。

一个代价。

而代价这件事有个特点：**它有时候不是“搞不成”，是“当下划不来”。**

两年之后，有八个人决定：划不来也干。因为他们在解决另一个问题的时候发现，那个环本身，才是真正的麻烦。

第二十章 减法

2017年，谷歌有八个人凑在一起写了一篇文章。这篇论文的标题后来成了这个领域最有名的一句话，翻译过来是：《注意力就是你所需要的一切》。

一句话，把之前二十年所有的主干都否了。

他们是怎么想到的？答案有点出人意料：**他们不是往前做加法，而是往回做减法。**

那半年，他们把当时最主流的翻译模型拆开，一件一件往下卸。每卸掉一件，就问两个问题：这件东西到底替我们做了什么？卸掉之后差的这一块，能不能从别的地方补回来？

卸到最后一层，只剩下了注意力。

而这时候他们发现，**那个被所有人当成主干的“环”，其实只干了两件事。**

第一件事：告诉模型这些字有先后。因为环是一个字一个字走的，走到第三个字的时候，前两个字的信息已经含在状态里了，所以“狗咬人”和“人咬狗”在环里的走法完全不同，模型自然分得清。

可这件事，本来就不必靠环来做。他们想了个很土的办法：既然环的作用只是让每个字身上带着它在第几位这个信息，**那直接把这个信息打上去不就行了吗？**

第二件事：让相隔很远的字能够互相影响。这一件才是环真正的看家本事——状态含在环里一路往下带，前面的字就能影响到后面的字。

可它带得越远越弱。前面说过，状态带过三十个字就淡得几乎看不见了，所以环的这件本事，其实一直是打折的。

到这里，那八个人停下来，问了一句很直白的话：既然环是靠转手来传递影响，而转手转得越远越弱，**那我们为什么不干脆让它们直接对上眼？**相隔再远的两个字，直接见面，不就行了。

很多技术的突破都是这样发生的。它不是发明了什么新东西，而是把一件所有人都以为必须如此的老规矩拿去问了一句“为什么”。**减法比加法难，因为加法只是增加，而减法需要先承认：**我们一直在背着这个东西，可能是多余的。

第二十一章 并流

他们想出来的东西，叫自注意力。

规矩很简单：把一句话里的每一个字，都拿去跟这句话里所有的字——包括它自己——对一遍。算出一串数，这串数说的是：此刻这个字跟句中每一个字各有多少关系。然后照着这串数，把句中所有的字各取一些，掺在一起，凑成这个字“看过全场之后”的新意思。

比方说句中有一个“它”字。自注意力算一遍之后，“它”这个新意思里，掺进去最多的会是它前面那个名词，其次可能是那个名词旁边的修饰，其余的掺得极少。这个“它”就不再是个孤零零的代词了，**它身上已经带上了它所指的那个东西。**

好处有两条，一条比一条重要。

第一条是距离。在环里面，两个字要互相影响，得一步步转手，隔三十个字就要转三十道手，每转一道淡一层；在自注意力里面，**任两个字之间永远只隔一次对看。**排第一的字要和排第一千的字互相影响，也是一步就到。

第二条更要命：并行。环是一步接一步走的，前一步不算完，后一步开不了口，所以一千块显卡只能用一块；自注意力不是，**所有字同时张口，同时互相看，同时算账，谁也不用等谁。**

想想这是什么意思。从 RNN 立派那天起，将近二十年里最大的那处死穴就是慢，就是不能并行。这二十年里，各路人马想尽办法去绕它：把句子切成两截分头处理再拼起来，把长句硬拆成几个短句，搞各种分层结构。而自注意力一出来，这个规矩连同那个环一起被请了出去。

一千块显卡，头一回能全部开动。

这就是它为什么会赢。不是因为它比环更聪明，而是因为它把一份谁也绕不过去的活，从“只能一个人干”变成了“可以一千个人同时干”。

话说回来，这个道理本身并不新鲜：用更多的机器干同样的活，前提是这份活能被切开。

环切不开。

自注意力天生就是切开的。

第二十二章 记号

但去掉环，立刻带来一个麻烦：**顺序信息没了。**

可以自己想一想：如果让一句话里每个字都跟所有字对一遍，而且所有字同时算——那“狗咬人”和“人咬狗”，在它眼里有什么区别？

没区别。

因为“对看”这个动作是同时发生的，谁先谁后它不知道。字还是那三个字，两两之间的关系也一样，算出来的结果自然一样。这就把语言最要紧的那点东西丢掉了。

怎么补？那八个人用了一个很土、但极其有效的办法：他们给每一个位置准备了一串特定的数字。位置1是这样一串，位置2是那样一串，位置3又不同，每一串都不一样，而且每一串规律是固定写死的，跟句子内容完全无关。然后把这串数字直接加到那个位置的字的表示上。

加完之后，字还是那个字，意思还是那个意思，只是每个位置的字身上多了一点只属于这个位置的记号。排在第二位的字走到哪儿，都带着“我是第二位”这个记号。这样一来，即使所有字是同时处理的，模型也能从这些记号里读出顺序。

这个东西叫位置编码。

我得说一句：它的做法看起来非常像作弊。直接往数据上加一串人为设计的数字，**这在数学上很不优雅**。但它管用，而且后来人们在这上面花了大量精力去改进——因为发现它的细节设计对模型影响很大，尤其是在长文本上的表现。

注意，这是那八个改动里，**唯一一个靠人手写进去的东西**，其余的都由网络自己学。而在接下来的几年里，这一处手写的设计，成了最常被人拿来改进的地方：有人换了新的算式，有人干脆让网络自己学位置，各种方案打了很多年。

故事讲到这里，可以总结一下这门技术在这一步完成的事情：**它把一种必须按顺序执行的任务，变成了可以同时执行的任务。**

听起来只是个工程优化。但整个七十年的历史里，几乎所有的大爆发都是这一类——**把不能并行的变成能并行的。**

AlexNet 能赢，是因为卷积让一张图的运算可以大规模并行；Transformer 能赢，是因为自注意力让一句话的运算可以大规模并行。

算力摆在那里，谁能让算力用起来，谁就赢。

第二十三章 众目

自注意力还有一个不够用的地方：**一组配方，一次只能算一种关系。**

可字与字之间的关系，从来不止一种。

举个例子：“他把我上次借给他的那本书还回来了。”这一句话里，同时存在好几种关系——“他”和“我”分别是谁，谁把书借给谁；“书”是“还”这个动作的对象；“那本”在修饰“书”；“上次”在限定时间。这是好几种完全不同的关系，缠在一句话里，一组配方算不过来。

那八个人的解法是：**不用算过来，多开几组便是。**

同一句话，同时开八组、十六组、几十组不同的配方，每组各管一样：这一组只问“这个代词指的是前面的谁”，那一组只问“这个动词的主语是谁”，又一组只管“相邻两个字怎么连”。每一组各算各的权重，各取一份结果，最后拼起来，当成这个字的新意思。

这个东西叫多头注意力。

它带来的好处是：同一个字身上，会同时带着好几种不同关系的信息。一个“它”字，既记住了它指的是谁，也记住了它在句子里是个什么成分，还记住了它跟旁边几个字是什么关系。可以把它想成：**一个字的身份不是一条，是好几条**，这几条由不同的“头”分别负责，最后合成一个完整的印象。

而且这些头分工的方式，不用人来规定，网络自己会学。给它看几百万篇文本，它自己会发现“管代词”这件事该单开一个头，没有人告诉过它要这么做。

这是又一个“**人只管形式，内容自己长**”的例子。从罗森布拉特那条线，到杨立昆的卷积，到这一手，走的都是同一条路。

第二十四章 代价

这套东西有一个天生的代价，而且这个代价后来成了它最大的麻烦。

句中每一个字都要看所有字。一句话有六十个字，对看的次数是六十乘六十，三千六百次；一千个字，就是一百万次。**句子长度翻一倍，计算量翻四倍。**

这就是前面巴达诺那句话说的意思——字一多就看过来了。它有一个很形象的说法，**叫平方的代价。**

为什么它是个麻烦？因为语言里的长文本太常见了。一份合同、一篇论文、一整本书、一段几百轮的对话，字数轻轻松上万。一万个字，对看次数是一亿；十万个字，是一百亿。

这就不是慢了，是根本算不动。

而**这是 Transformer 唯一的一个原生缺陷**。注意“唯一”这个词——它别的方面都在进步：并行性更好、长距离依赖更强、能堆得更深、能通吃各种模态。唯独这个平方的代价，是它从娘胎里带出来的，后面几年所有的重要改进，几乎都在跟它较劲。

有的让大部分对看只发生在邻近的字之间，远处的少看；有的把计算过程拆成小块，算一块收一块，不让内存爆掉；有的干脆放弃精确的对看，用一个近似的办法算。

但这些都是把账往后推，没有一笔真正把这个平方抹掉。要真正抹掉它，得换一条路。这条路，故事后面会说。

不过在当时——2017年——这个代价并不显眼。因为那几年大家能处理的句子还很短，几百个字，平方一下也还扛得住。

等到人们开始往模型里塞整本书的时候，才知道这笔账有多贵。

第二十五章 定形

2017年那篇论文发出来之后，圈子里没什么反应。真的，那一年几乎没有人当回事。

原因也简单：它在当时的成绩没有明显超过原有方案。用 RNN 加注意力的模型，在翻译任务上表现也很好；而 Transformer 结构更复杂，学起来更费劲，训练还更贵。所以2017年，它是一篇“挺有意思但看不出必须用”的论文。

真正炸开是在2018年。

这一年出了两个东西。一个叫 **BERT**，用的是 Transformer 的编码器部分，专门练“把句子挖空让人填”的活，它证明了这套结构练出来的语言理解能力比之前所有方法高出一大截。另一个叫 **GPT**，用的是解码器部分，专门练往下续写，第一代只有一亿一千七百万个参数。

这两个东西一出来，整个领域都静了。

然后，接下来发生了一件历史上极少见的事：**各家的模型结构，忽然长得一模一样了。**

翻开2018年之后的论文，拆开任何一个大模型，骨架都是同一套：自注意力、多头、位置编码、残差连接、层归一化。甲家的模型和乙家的模型，差别只在层数多少、头开几组、位置编码怎么算这些细节上。

主干那几根梁，谁也没有动。

这在历史上是很反常的。从1957年到2017年，六十年里主流结构每三五年就要换一次：感知机、多层网络、卷积、残差、循环、注意力，一个接一个。

为什么到这一代就停了？

因为这一次，该凑齐的都凑齐了：能并行，因为自注意力天生可切；任意两个字能直接相连，不必转手；顺序信息可以补回来，靠位置编码；能堆得深，靠残差连接；能通吃各种数据，因为文字、图像、声音都能切成序列。

一个结构，把前面六十年所有的坑都填上了。

填完之后，就没有“换个结构”的必要了。剩下的问题是另一个——

这个结构，能有多大？

这个问题在2020年有了答案。而这个答案，让整个领域的玩法彻底变了。

第二十六章 曲线

2020年，一位叫贾里德·卡普兰的人，做了一件非常枯燥的事。

他把近几年各家训练模型的记录，能收到的全都收来：哪家的模型有多少参数、喂了多少数据、烧了多少算力、训练完是什么水平，一条条抄下来，画在一张大图上。

画完他自己也吃了一惊。图上那些点没有散成一团乱麻，它们落在几条非常规整的曲线上。

原来模型的水平跟三样东西直接挂钩：参数有多少、数据有多少、算力有多少。这三样同时往上加，错误率就按一条极其稳定的曲线往下走。加十倍掉多少，加百倍掉多少，都有定数。

这条曲线不是直线，它掉得越来越慢，但它从来不回头。

这就是它最要命的地方：**你加得多，它就一定更强，而且强多少，能算出来。**

这个东西叫 Scaling Law。我一直想给它找个中文名，最后觉得“规模定律”太平淡了，不如叫“堆得越多、越好、而且能算”定律。

这张图带来的变化有多大？这么说吧，**它改变的不是技术，是做事的方式。**

以前做这件事，是赌。你花了三年，烧了几千万，训出一个模型，谁也不知道成不成；成了是运气，不成也是常事。所以那个年代最值钱的人是巧人——能想出新花样的人。

有了规模定律之后，**做事变成了投。**你要多少成绩，就先算需要多少参数、多少数据、多少算力，照着单子去凑。

成绩不再是赌出来的，是买来的。

于是整个领域的风气翻了过来。从前比的是谁的法子巧，现在比的是谁的家底厚；从前各家争的是“想得到”，现在争的是“凑得齐”。

能凑齐的人，比能想通的人阔气得多。

这话说得刻薄，但那就是那几年的实情。

说实话，我第一次看到这条定律的时候，心里是有点不舒服的。因为它意味着：**在结构已经定型的前提下，聪明这件事变得没那么重要了。**一个天才想出一个新结构，可能提升三个百分点；而另一个人什么新东西都没想，只是把算力加十倍，提升十个点。

前者写进论文被人称赞，后者写进财报被人追捧。

这两件事在2017年之前是不成立的。是2020年这张图，让第二件事变得成立。

第二十七章 三个数字

接下来的三段故事，按年份讲。

2018年，一亿一千七百万。这是 GPT 第一代，用的还是 Transformer 那套，没添什么新花样。放在今天看，一亿一千七百万参数跟个玩具差不多。

2019年，十五亿。GPT-2 出来了，规模是上一代的十几倍。有个细节挺能说明当时的气氛：OpenAI 一开始不敢把它完整放出来。他们的顾虑是，这东西可能被用来批量生成假新闻、做钓鱼邮件，所以先放了一个小版本，大版本拖了几个月才公开。

这在今天听起来有点像笑话。

2020年，一千七百五十亿。GPT-3。

从一亿到一千七百五十亿，两年，涨了一百五十倍。

他们是怎么做到的？说出来有点让人泄气：**什么新方法都没用，就是把料备得更足。**备的是什么料？网上能抓到的文本、书籍、百科、代码、论坛帖子，反正有字的都要，加起来几千亿个词。烧的是什么？几千块 GPU，连着烧了几个月，电费加机时，几百万美元。

然后，出了一件让所有人没有预料到的事：**它学会了很多没有人专门教过它的东西。**

以前做一个任务，流程是这样的：我要让它认垃圾邮件，那就准备十万封标注好的邮件，重新训练一个模型；我要让它做翻译，那就准备平行语料，重新训练。换一个任务，就要重训一次。

GPT-3 不是这样。**你给它几个例子，它就照办了。**

比如在输入里写：

把下面这句话改得更客气：“东西还没到。” 改成：“麻烦您看一下，东西好像还没发出，能帮我查一下吗？”

把下面这句话改得更客气：“这个不行。” 改成：

它就会自己接下去，给你一个客气的版本。

你没有教过它“改客气”这个任务，你只是在题目里举了两个例子。

这个现象后来被叫做上下文学习。它意味着：**不需要为每一个任务单独训练一个模型了**，只需要在一个已经训练好的模型上，把要求说清楚。

这一下，整个领域的做法变了。以前大家说的是“我要训一个认脸的模型”，从那以后，大家说的是“我要拿这个模型去认脸”。

这两句话听着差不多，其实是两回事。**前者你要养一个孩子，后者你只是使唤一个已经养大的。**

第二十八章 通吃

GPT-3 出来那年秋天，另一件事也发生了。

有人对着图像想了件“大不敬”的事。这人原先做视觉的，后来去了谷歌。他盯着 ImageNet 那一千四百万张图，问了一句：**Transformer 既然能处理文字，为什么不能处理图像？**

同行都笑他。因为**图像这块地，是卷积网络守了三十年的**。从杨立昆想通那件事开始，“局部感受野加权重共享”这套先验就是视觉的立身之本——一只猫的耳朵，长在画面左边还是右边，都还是耳朵。这套先验被直接写进了结构里，省下了大量数据。

而 Transformer 不懂这套先验。它只知道“每个元素跟每个元素对看”。

这个人偏要试。

他做的第一件事，是把图切了。一张图，横竖切成十几块、几十块小方块，每一块当成一个“字”来用。原先看图是一格一格的像素，现在看图是一句一句的方块。切完打上位置编码——图本来没有先后，那就按切块的位置人为规定一个先后——然后把这串“方块字”整句喂进 Transformer。

第一年，输得很难看。因为 ImageNet 那一千四百万张图，跟 Transformer 平时吃惯的几千亿词文本比，实在是太少了；而它又不像卷积那样天生知道“挪位置没关系”，这些事它得从头学一遍。数据不够，就学不动。

第二年，数据备足了，再练，打平。

第三年，反超。

这个东西叫 ViT。

它带来的震动，比它本身的成绩更大。因为这意味着：Transformer 不但能吃文字，还能吃图；而稍后人们就发现，**它还能吃声音、吃乐谱、吃蛋白质结构、吃一切能被切成序列的东西。**

原来 Transformer 不是一个“处理语言的模型”，**它是一个通吃的骨架。**

有一件事值得记下来。据说杨立昆听说这事之后，并没有不高兴。他在一个场合说了两句话，我觉得很有分量。

第一句是：**切块这件事，我当年就该想到。**

第二句是：**我那一套先验，往后不必写在结构里了——数据够多的话，它自己会学到。**

他用三十年时间，把一句话从“人得写进结构”，讲到了“人不写，它自己也会”。

而这句话，从1957年罗森布拉特放下那本人眼构造手册那天起，就一直在被反复验证。

第二十九章 野马

GPT-3 出来了，什么都能干。但所有用过它的人，都会先经历一段困惑期。

因为它不配合你。

你问它：“帮我把下面这句话改得客气一点：这个东西还没到。”

它给你的可能是：

“帮我把下面这句话改得客气一点：这个东西怎么还没到。帮我把下面这句话改得客气一点：这个.....”

它在接着往下编。你写的那句话，被它当成了文章的开头，而不是一个要求。

再看一个更典型的。你问它：“法国的首都哪里？”它可能回答：“德国的首都哪里？意大利的首都是哪里？”

它在给你出题，因为你给它的就是一道题的格式。

这个毛病背后的原因，得回到它是怎么练出来的。**它这辈子只被训练过一件事：看了前面这么多字，下一个字应该是什么。**训练它的时候，数据是从网上抓来的文章、书、帖子，它看到的是“.....所以说，法国的首都是巴黎”这样一整段文字，然后被要求预测“巴”“黎”这些字。

它从来没有学过“有人问了一句话，我应该回答他”这件事。因为网上的文章里没有这种场景，文章是写给人看的，不是对话。

可以这样想象它：**一个读遍了天下所有书的人，但他这辈子没有跟任何人说过一句话。**你问他一个问题，他脑子里第一个蹦出来的念头不是“我该怎么回答他”，而是“书上这句话后面通常接什么”。所以他要么接着你的话往下编，要么给你蹦出一堆相关但没用的东西。

这个毛病很致命，因为它很能干，却没法被使用。你想让它干活，得先教会它“配合”。

注意，这是另一个层面的问题。前面几十年，这个领域一直在解决能力问题——能不能认对、能不能译好、能不能更准。而现在出现了一个新的问题：**能力有了，但它不听指挥。**

这需要新的方法，而且这个方法跟以前所有的都不同。

第三十章 缰绳

第一道工序很简单，也很贵：请人写。

请一批人，写几万条“问题加标准回答”。问题就是用户真会问的那些，回答就是人写下来的、规矩得体的答法。比如：

问：把这个句子的语气改得客气一点——“东西还没到。” 答：麻烦您帮忙看看，东西好像还没发出，能帮我查一下吗？

问：我朋友借了我三千块钱一直没还，我该怎么开口？ 答：可以从这几个角度考虑……

几万条这样的东西，写出来，然后拿这些去接着训练那个大模型。

这一步调完之后，它就从“续写”变成了“回答”。再问它“改得客气一点”，它不会接着编了，它会给你一个客气的版本。

这个东西叫指令微调，早年的术语叫 SFT。

但是，这一步只能教会它格式，教不会它好坏。

为什么？因为教它的时候，给的是一份标准答案。而一个好问题，好的回答有无数种，烂的回答也有无数种。你给了它一种好的，它学会了“合格的回答长什么样”，可它不知道在这个基础上还能怎么变得更好。

举个不太恰当的例子：你能通过示范教会一个学徒怎么把菜做熟，但你很难通过示范教会他怎么把这道菜做到让客人惊艳。**因为“惊艳”这种东西，很难用一份标准答案表达出来；**而“不惊艳”和“难吃”之间，又是一大片灰区。

更要命的是，请人写答案的时候会发现一个尴尬的事实：同一道题，十个人写出十份不一样的答案，而这十份答案的质量其实是参差不齐的。

那模型学到的是什么？**是这十份的平均。**

平均下来，就是一个谁也挑不出错、但谁也不觉得好的东西。

所以这一步只能做到及格线。要往上走，得换一个思路：**不去告诉它什么是“对的”，而是告诉它什么是“更好的”。**

第三十一章 褒贬

第二道工序分三步。先把这三步的分工说清楚，因为这套方法本身很巧妙。

第一步，让它出题，人排高低。同一个问题，让模型一口气生成七八个不同的回答，然后请人来排个序：这个比那个好，那个比第三个好。

注意这里的关键——**人不写标准答案**，人只做一件事：看这几个回答，说哪个更好。

为什么这样更省力？**因为判断比创作容易**。让一个人写一篇好文章，他得憋半天；让他看两篇文章说哪篇好，十秒钟就给你答案。用同样的成本，这一步能换来多得多的信息。

第二步，用这些排序，训一个“打分器”。现在手上有一堆数据，长这样：问题是某个问题，回答A，回答B，结论是A比B好。拿这些东西去训练一个专门的模型，它只干一件事：给它一个问题和一个回答，它给打个分。

这个东西叫奖励模型。

第三步，用打分器去指挥大模型。大模型每生成一个回答，打分器给个分，大模型就照着“怎么让这个分变高”去改自己。改上几万次、几十万次，它就越来会产出得分高的回答。

这三步合起来，叫 RLHF。

这里有一件事特别值得说说。注意第二步：训练打分器的时候，用的是“问题加回答，对应一个分数”这样的配对数据。

这叫什么？**这就是最老的那套监督学习。**

也就是说，这套最新潮、最花哨的对齐方法，最里头那一步，用的还是七十年前罗森布拉特那条线——给一堆输入配上答案，让模型去拟合。

七十年了，它还在那儿。

2022年11月30日，这一整套东西做齐、做得足够好用，放出来了。

叫 ChatGPT。

上线五天，一百万人；两个月，一亿人。**这是人类历史上用户增长最快的应用，没有之一。**

而且有件事特别有意思：**技术上讲，ChatGPT 并没有发明什么新东西**。Transformer 是2017年的，GPT 是2018年起的，指令微调和 RLHF 也都是2022年之前就有的。它赢在了把这几样东西拼齐了，而

且拼得足够顺手。

一个能陪你聊天、帮你写东西、你说人话它就能听懂的东西——这才是普通人第一次真正摸到人工智能。

七十年走到这里，才终于让一个不懂技术的人也能用上它。

这里要说一句公道话：这个领域的绝大多数贡献者，一辈子都在盼着这一天。他们做的是数学、是结构、是优化器，一篇文章写出来只有几百个人看得懂；而他们真正想做的事，是让一个普通人能跟机器说上话。

这个愿望在2022年11月30日成了。

尾声 像与真

故事讲到这里，本该结束了。

主线已经闭合：**那条线长成了能跟人说话的东西，而且第一次让不懂技术的人也能用上它。**这是一部史话能给出的最好的结局。

但有一件事必须留在这里说清楚，因为它不属于“已经解决的问题”那一列。

ChatGPT 出来之后，所有人都会经历同一个阶段：**先是惊叹，然后是疑惑，最后是被坑。**

因为它会一本正经地编。

你问它一本书里的一句话，它答得有条有理，连页码都报出来，而那本书里根本没有这句话。你问它一个法律问题，它引了一条法条，措辞无懈可击，而那条法条从来没有人定过。你让它算个数，它写出五步推理，每一步看着都对，最后那个数是错的。你让它介绍一个人的生平，它给你编出一段非常合理的经历，而这个人是他编出来的。

这件事叫幻觉。

先说清楚它不是什么：**它不是故意骗人。**它根本没有“骗”这个动作。

回到它是怎么练出来的，就明白了。它这一辈子只回答过一个问题：看了前面这么多字，下一个字该是什么。而这个问题问的是“最像样的”下一个字，不是“对的”下一个字。

它的目标是想，不是真。

所以它身上没有一本账，记着“我会什么”和“我不会什么”。你问它一桩它根本不知道的事，它不会停下来，它照样照着最像的样子往下写——而“像”的写法恰恰是：**书名要像书名，页码要像页码，法条要像法条。**

于是它把一整套最像样的东西摆到你面前，里头没有一样是真的。

这就是幻觉最麻烦的地方，也是它为什么难治：这不是某个可以补的小漏洞，**它是那道训练题本身带来的。**

而这正好把这本书拉回到它的起点。

1957年，罗森布拉特说的那句话是：我不去描述“看见”这件事，我直接造一台会看的东西出来；**我不管道理，先让它自己长出道理来。**

六十多年过去了，它确实长出了道理，而且长出了一整套没有人能完全看懂的、关于这个世界的道理。但它长出的道理一直是关于“像”的。

至于“真”——**它会不会有一天也长出关于真的道理，还是说“真”这件事本来就不在它那条路上，这本书回答不了。**

我们能确定的是：从那条线开始的那个问题，到今天也没有问完。

这本书讲的是前面的六十多年。后面的，得由别人接着写。